Class Object and Constructors

19 August 2026 17:03

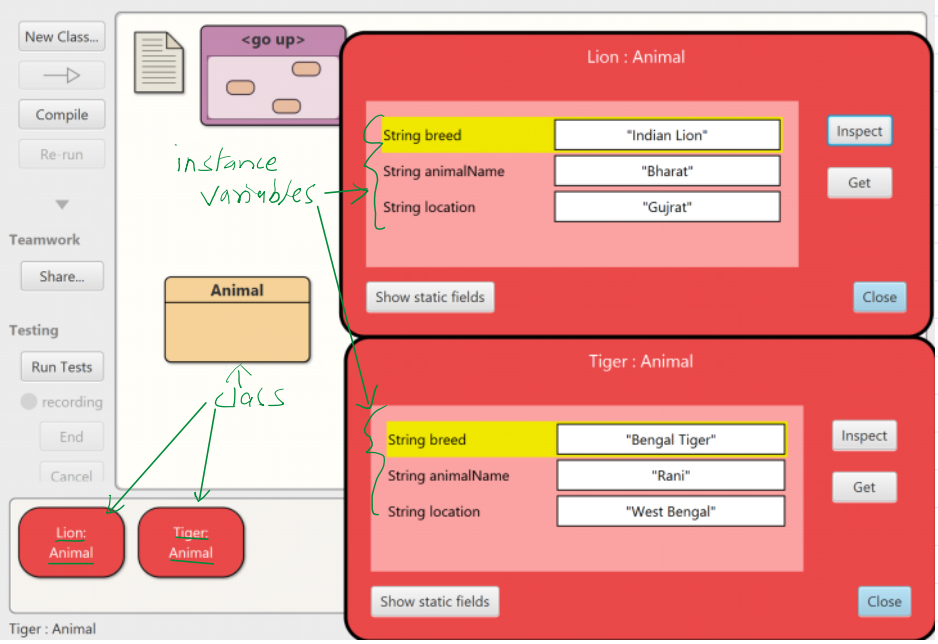
In object oriented programming, classes and objects play. Vital role in programming. These are two main pillars of OOP (object oriented programming). Without classes and objects we cannot create program in Java.

What is a class in Java?
A class is a group of objects that have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A Java class contains:

- Fields
- Methods
- Constructors
- Blocks
- Nested class and Interface

Fields: Variables stated inside a class that indicate the status of object formed from that class are called fields. Sometimes referred to as instance variables. They specify the data that will be stored in each class object. Different access modifiers such as public, private and protected can be applied to fields to regulate their visibility and usability.



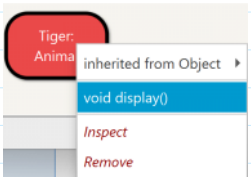
Methods: Methods or functions defined inside a class that include the actions or behaviors that object of that class are capable of performing. These techniques allow the outside world to function and change the object's state (fields) Additionally, methods can be void (that is they return nothing) or have different access modifiers. They can also return values.

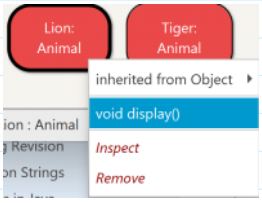
```
public class Animal
{
    String breed;
```

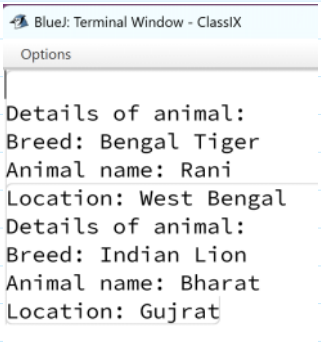
```
    String animalName;
    String location;

    public void display(){
        System.out.print("\nDetails of animal:");
        System.out.print("\nBreed: " + breed);
        System.out.print("\nAnimal name: " + animalName);
        System.out.print("\nLocation: " + location);
    }
}
```

Output:







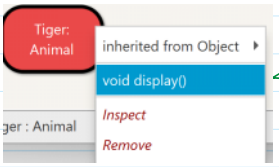
Constructors: Constructors are unique methods that are used to initialize class objects. When an object of a class is created using the new keyword, they are called with the same name as the class. Constructors can initialize the fields of an object or carry out any additional setup that is required when an object is created.

Blocks: Within a class, Java allows two different kinds of blocks: instance blocks, commonly referred to as initialization blocks and a static blocks. Static blocks, which are usually used for static initialization, are only executed once when the class is loaded into memory. Instance blocks can be used to initialize instance variables and are executed each time A class object is generated.

Nested Class and Interface: Java permits the nesting of classes and interfaces inside other classes and interfaces. The members (fields, methods) of the enclosing class are accessible to nested classes which can be static or non-static. Nested interfaces can be used to logically group

related constants and methods together because they are implicitly static.

What is an Object in Java?
An object is a real - world entity that has state and behavior. In other words, an object is a tangible thing that can be touch and feel like a car or chair, etc. The banking system is an example of an intangible object. Every object has a distinct identity. Which is usually implemented by a unique ID that the JVM uses internally for identification.



← This is the behavior

Real world objects: Pencil, Apple, Book, Bag, Board

Characteristics of an Object
- **State:** It represents the data (value) of an object.
- **Behavior:** It represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An objects identity is typically implemented via a unique ID. The ID value is not visible to the external user. However it is internally used by the JVM to identify each object uniquely.

For example, a pen is an object. Its name is Reynolds, its color is white, Known as its state. It is used to write, so writing is its behavior.

An object is an instance of a class. A class is a template or blueprint from which objects are created. So an object is the instance (result) of a class.

Object definition
- **An object is a real world entity.**
- **An object is a runtime entity.**
- **The object is an entity that has a state and behavior.**
- **Although the object is an instance of a class.**

Syntax of an Object:
<class name> variable_name or object_name = new <class name> ();

"new" keyword in Java: The ***new*** keyword is used to allocate memory at runtime. All objects get memory in the heap memory area.

In Java, an instance of a class (also referred to as an object) is created using the new keyword. The new keyword dynamically allocates memory for an object of that class and returns a reference to it when it is followed by the class name and brackets with optional arguments.

Initializing an object in Java

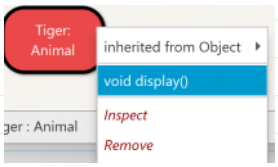
i. By reference variable.
ii. By Method

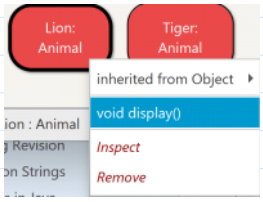
iii. By Constructor

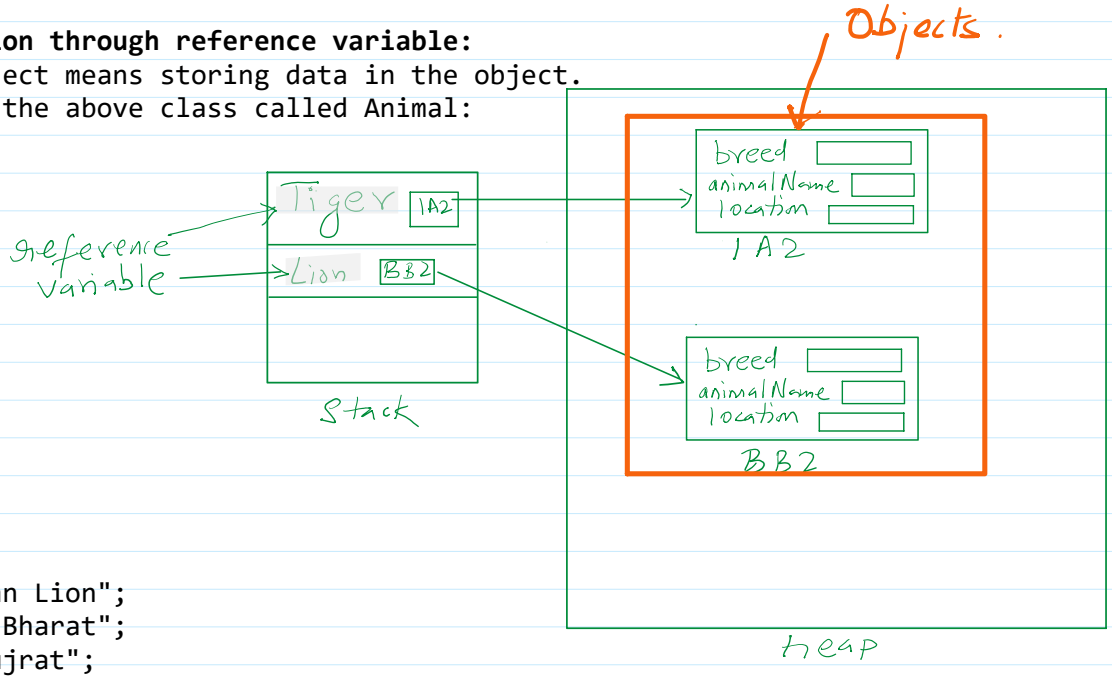
Object initialization through reference variable:

Initializing an object means storing data in the object.

For example: Using the above class called Animal:








```
Lion.breed = "Indian Lion";
Lion.animalName = "Bharat";
Lion.location = "Gujrat";
Tiger.breed = "Bengal Tiger";
Tiger.animalName="Rani";
Tiger.location = "West Bengal";

Lion, and Tiger is a reference variable.w
```

Object Initialization Through Method

```
package ClassAndObjects;
import java.util.*;


/**
 * Write a description of class Animal here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Animal
{
    class Location{ //Nested class
        int latitude, longitude;
    }
    {
        System.out.print("\fHello!\n\n");
    }
    String breed;
    String animalName;
    String location;

    //setter methods
```

```
    //getter methods

    public void insertRecord(){
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter breed: ");
        breed = sc.nextLine();
        System.out.print("Animal name: ");
        animalName = sc.nextLine();
        System.out.print("Location: ");
        location = sc.nextLine();
    }

    public void display(){
        System.out.print("\nDetails of animal:");
        System.out.print("\nBreed: " + breed);
        System.out.print("\nAnimal name: " + animalName);
        System.out.print("\nLocation: " + location);
    }

    public static void main(String[] args){
        Animal a1 = new Animal();
        Animal a2 = new Animal();
        Animal a3 = new Animal();

        a1.insertRecord();
        a2.insertRecord();

        a3.insertRecord();

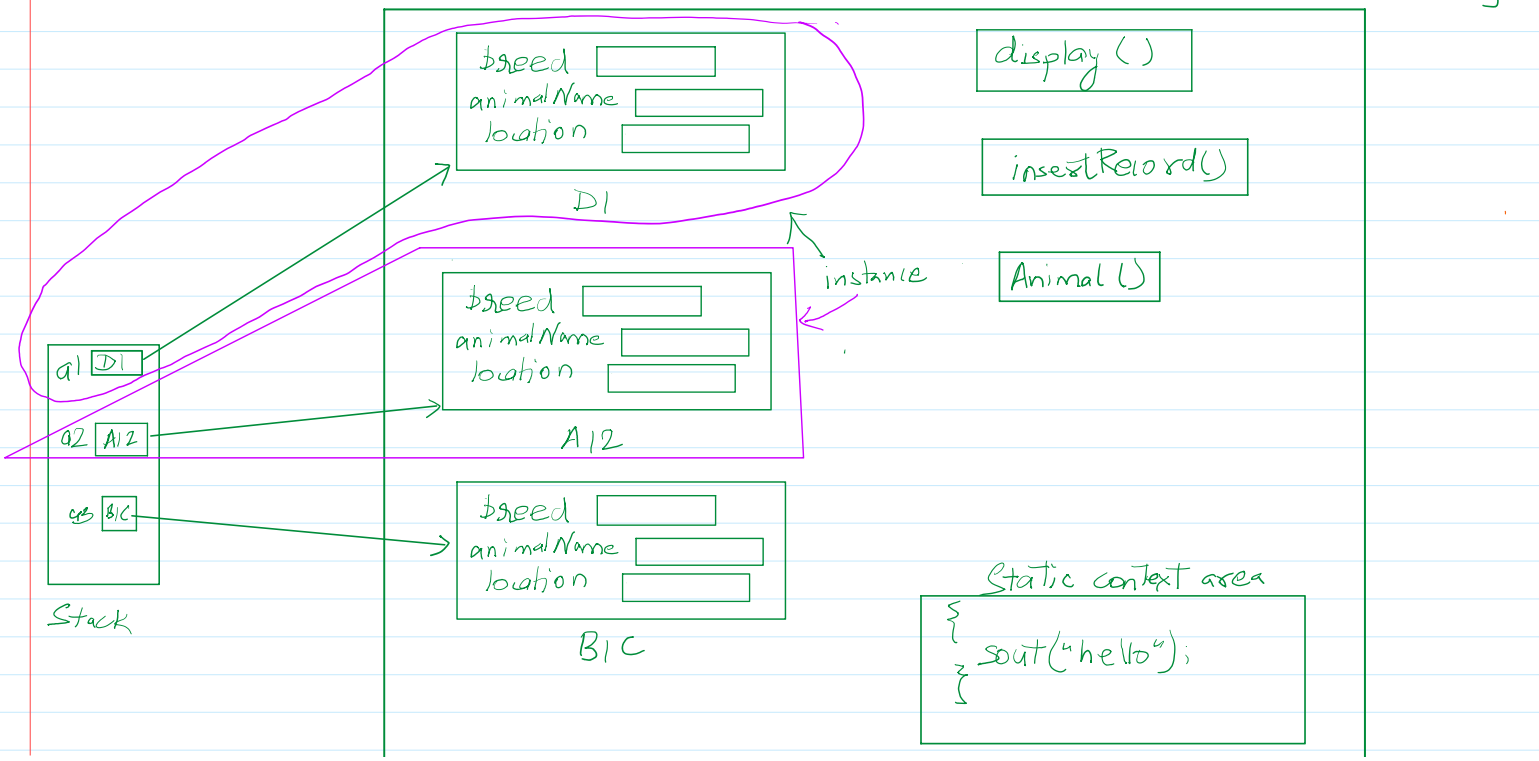
        a1.display();
        a2.display();
        a3.display();
    }
}
```

1. Allocate memory during runtime and returns reference of the object to the reference variable.

Animal a1 = new Animal();

constructor created by JVM

2. keyword is calling this constructor to initialize the object



heap

Reusability of code during runtime.

Output:

